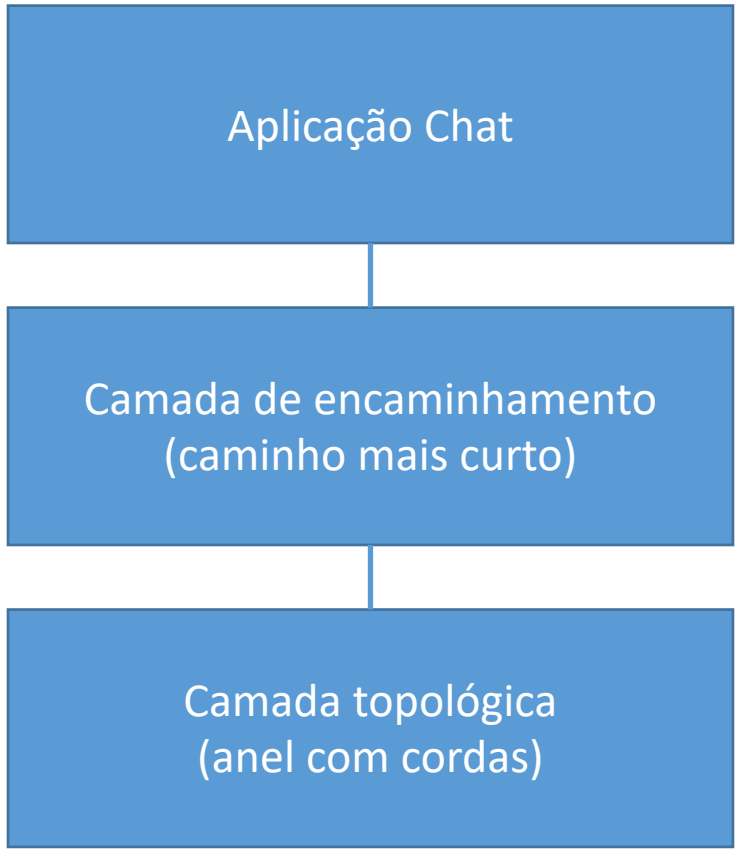
RCI 2023/2024
Chat Sobre Anel

Chat sobre anel

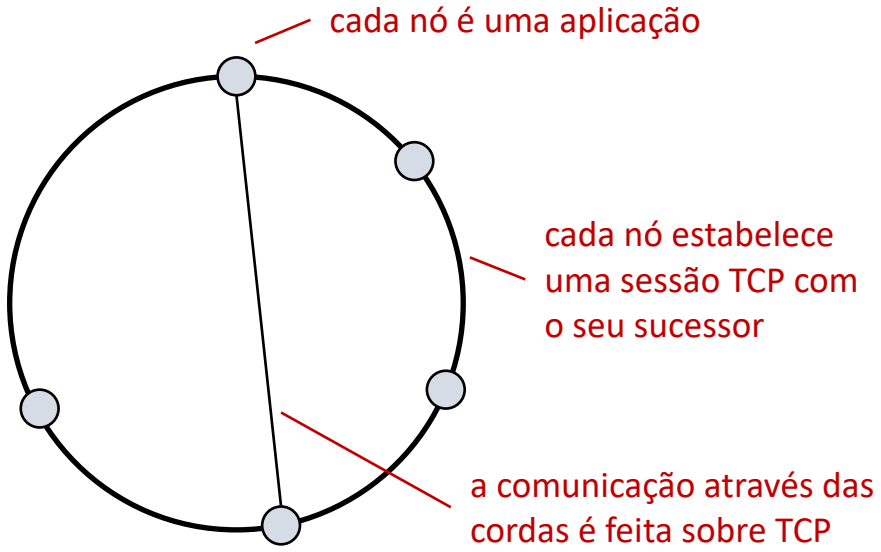


A aplicação permite a troca de mensagens entre nós de uma rede.

O encaminhamento é por caminho mais-curto. O protocolo de encaminhamento mantém em cada nó uma tabela de encaminhamento, a partir da qual se calcula a tabela de expedição que indica para cada destino qual dos seus vizinhos está mais próximo desse destino. Ele reage à adição de uma adjacência e à remoção de uma adjacência.

A rede que interliga os nós tem como base um anel que circula sobre todos. A este anel podem adicionar-se cordas. O protocolo topológico mantém o anel aquando da entrada de um novo nó e da saída de um nó.

Concretização da aplicação



Invocação da aplicação

```
COR IP TCP regIP regUDP
```

nome da aplicação

endereço da máquina que aloja a aplicação

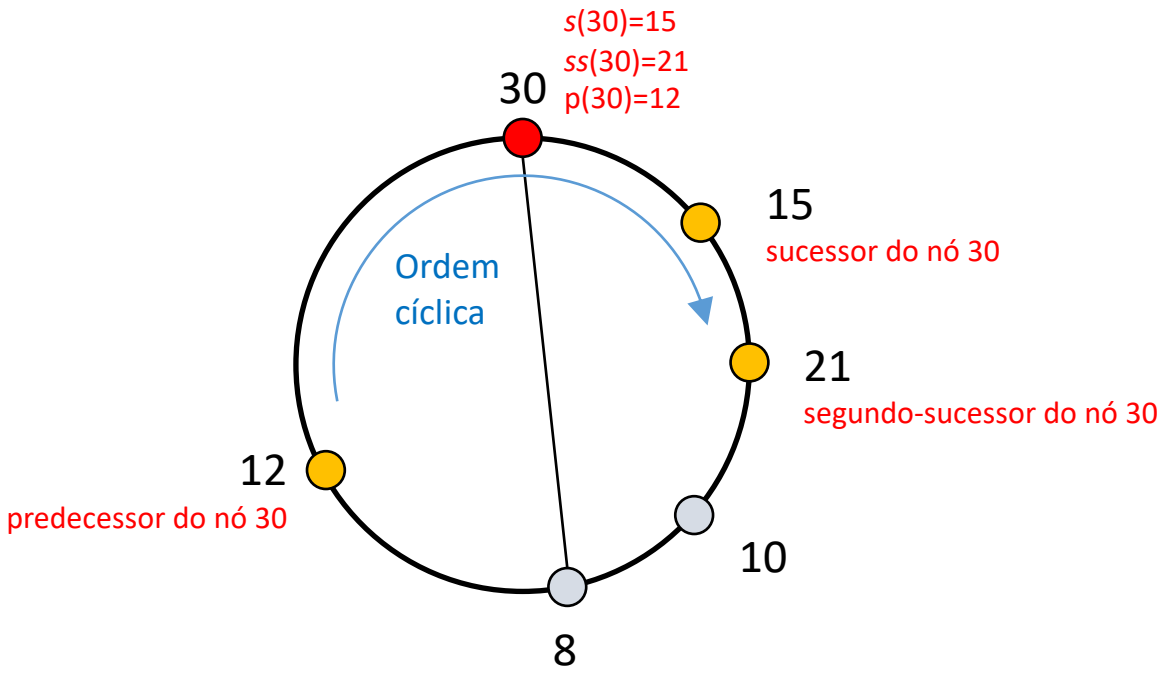
porto TCP servidor da aplicação (porto de escuta)

Os parâmetros regIP e regUDP são o contacto, endereço IP e porto UDP, respetivamente, do servidor de nós fornecido pelo corpo docente. Por omissão, estes parâmetros tomam os valores 193.136.138.142 e 59000.

Os valores de ***ring*** são representados por três dígitos, podendo variar entre ***000*** e ***999***; os valores de ***id*** são representados por dois dígitos, podendo variar entre ***00*** e ***99***.

Comando da interface	Descrição
join (j): **j *ring id***	Entrada de um nó com identificador ***id*** no anel ***ring***.
direct join (dj): **dj *id succid succIP succTCP***	Entrada de um nó com identificador ***id*** diretamente num anel, sem consulta ao servidor de nós.
chord (c): **c**	Estabelecimento de uma corda para um nó do anel que não o sucessor ou o predecessor. Cada nó estabelece no máximo uma corda.
remove chord (rc)	Eliminação da adjacência associada à corda.
show topology (st)	Mostra o estado do protocolo topológico, consistindo nos identificadores e contactos de: (i) ele próprio; (ii) o seu sucessor; (iii) o seu segundo-sucessor; (iv) o vizinho na corda que criou, se houver, e no identificador do seu predecessor.
show routing (sr): **sr *dest***	Mostra a entrada da tabela de encaminhamento de um nó relativa ao destino ***dest***.
show path (sp): **sp *dest***	Mostra o caminho mais curto de um nó para o destino ***dest***.
show forwarding (sf)	Mostra a tabela de expedição de um nó.
message (m): **m *dest message***	Envio da mensagem ***message*** ao nó ***dest***.
leave (l)	Saída do nó do anel.
exit (x)	Fecho da aplicação.

Camada Topológica: anel com cordas

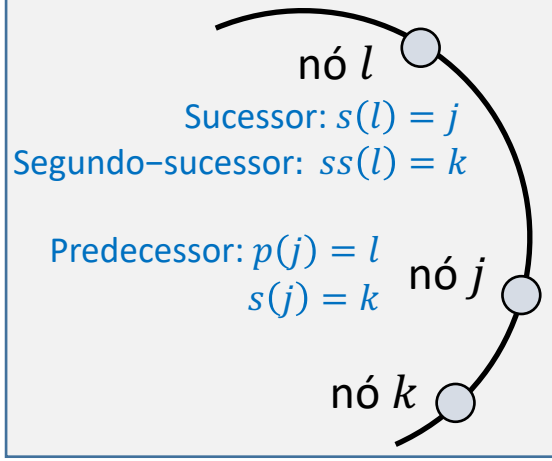


- Cada nó tem um identificador único.
- Para efeitos de manutenção do anel, este está orientado no sentido dos ponteiros do relógio.
- O sucessor de um nó é o próximo nó do anel percorrido no sentido dos ponteiros do relógio e o predecessor de um nó é o nó que o tem como sucessor.
- O segundo-sucessor de um nó é o sucessor do seu sucessor.
- Cada nó mantém atualizados os identificadores e contactos do seu sucessor e segundo-sucessor, bem como o identificador do seu predecessor.

Há um **servidor de nós** (fornecido pelo corpo docente) onde estão registados os identificadores e os contactos de todos os nós pertencentes ao anel a cada momento.

Camada Topológica: Entrada do nó i sabendo o contacto do successor j

Entrada do nó i ficando com o nó j como seu sucessor

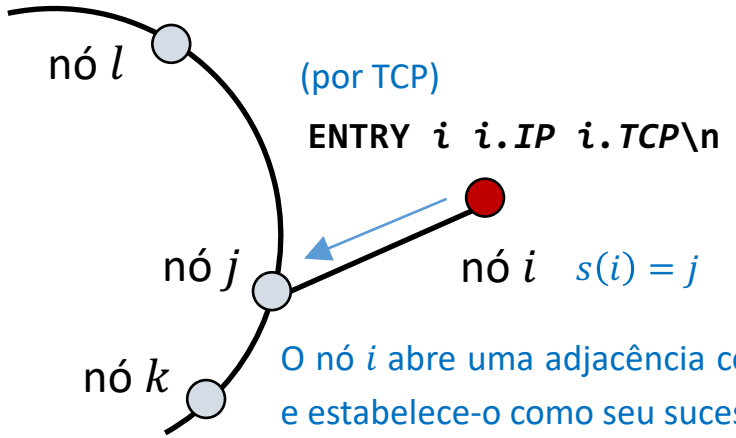


interface de utilizador

> **dj *i j j.IP j.port***

nó i

$s(i)$ é conhecido do utilizador e especificado no comando **dj**.

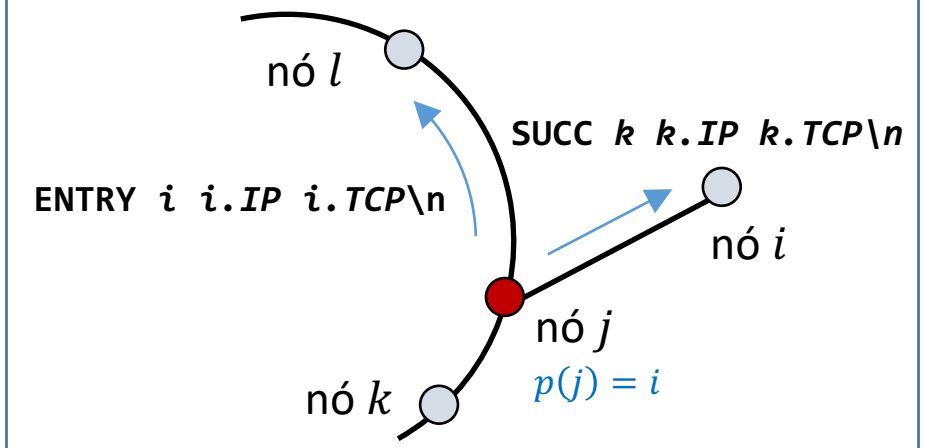


(por TCP)

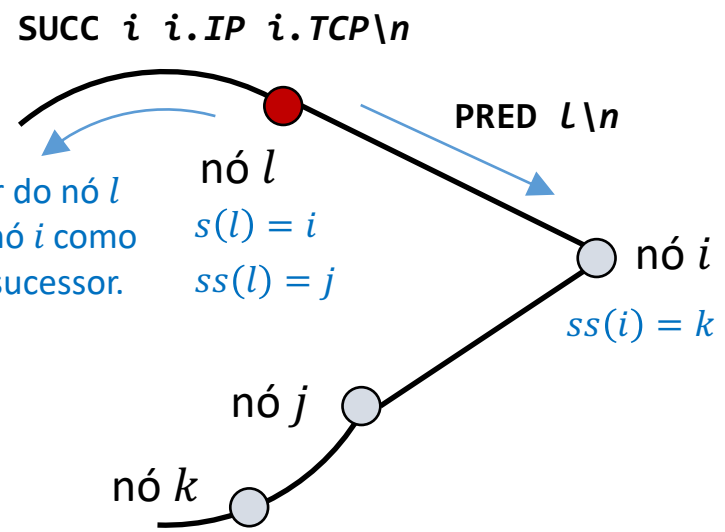
ENTRY *i i.IP i.TCP*\n

nó i $s(i) = j$

O nó i abre uma adjacência com o nó j e estabelece-o como seu sucessor.



O nó j informa o nó i da existência do nó k, informa o nó l da entrada do nó i e atualiza o seu predecessor para i.



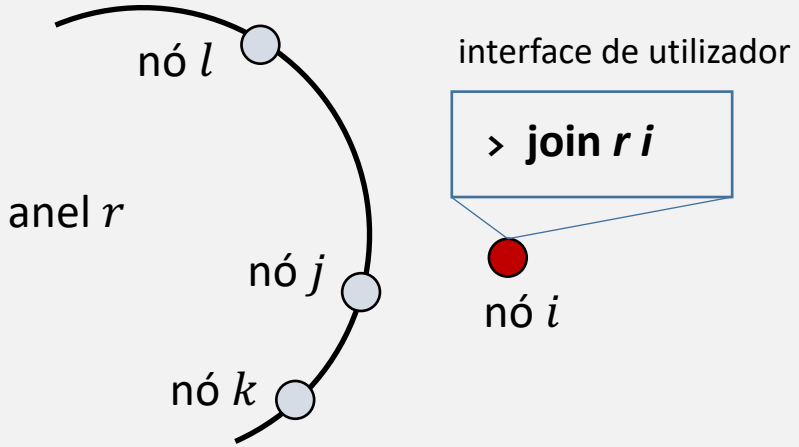
O predecessor do nó l estabelece o nó i como seu segundo-sucessor.

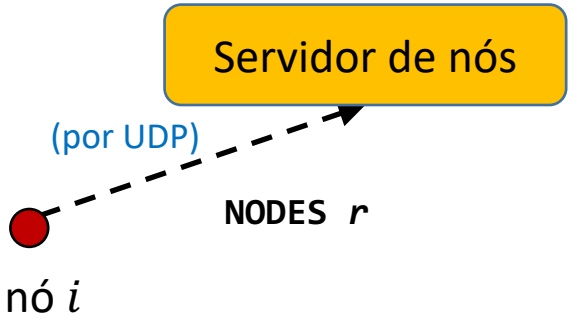
O nó i atualiza o seu segundo-sucessor para k .

Concorrentemente, o nó l fecha a adjacência com o nó j , abre uma adjacência com o nó i , informa-o de que será seu predecessor, informa o seu predecessor da entrada do nó i , atualiza o seu sucessor para i e o seu segundo-sucessor para j.

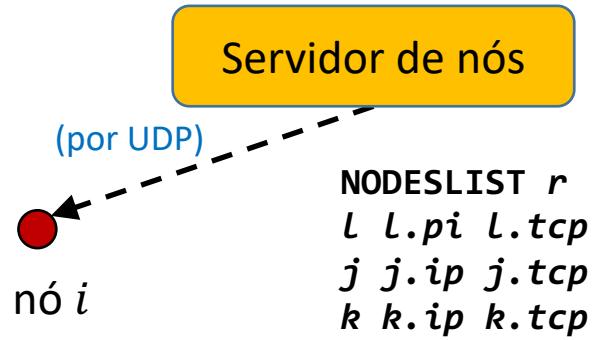
Camada Topológica: Entrada do nó i no anel r

Entrada do nó i no anel r

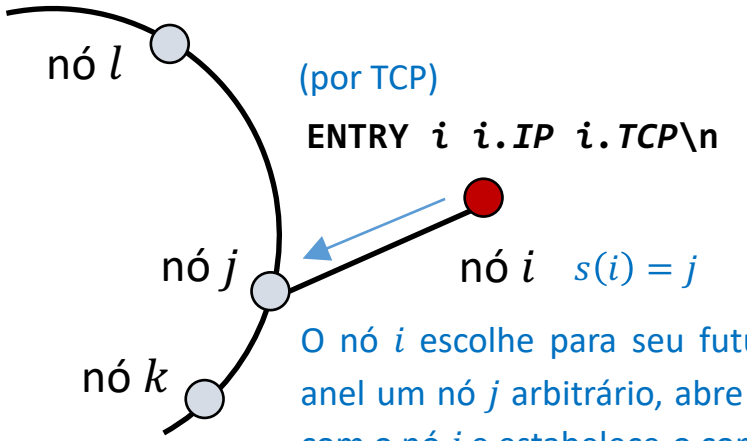




O nó i pede ao servidor de nós o identificador e respetivos contactos dos nós existentes no anel r.

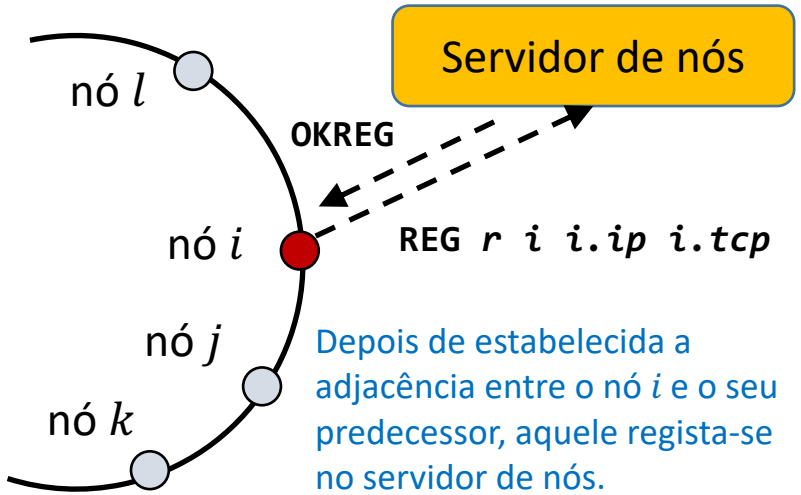


O servidor de nós envia uma lista com uma linha por nó no anel r. Cada linha dessa lista contém o identificador e contacto de um nó.



O nó i escolhe para seu futuro sucessor no anel um nó j arbitrário, abre uma adjacência com o nó j e estabelece-o como seu sucessor.

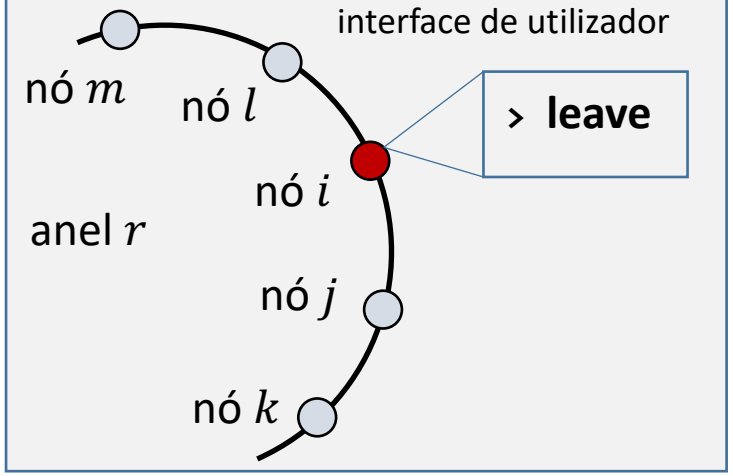
(ver slide anterior)

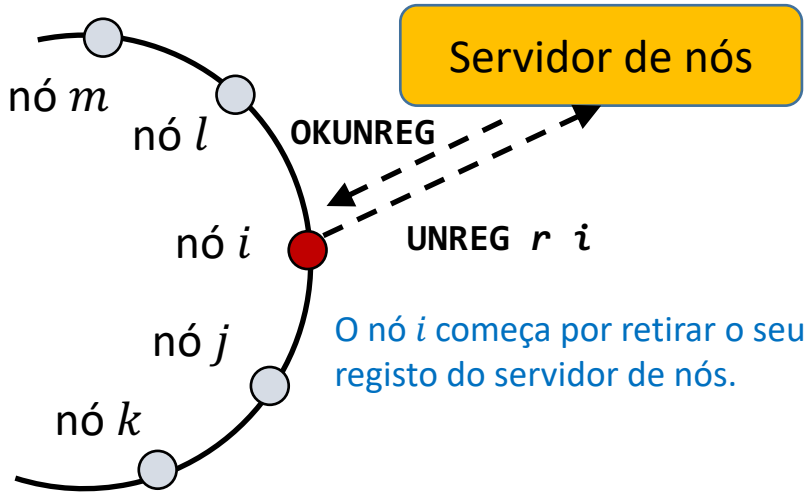


Depois de estabelecida a adjacência entre o nó i e o seu predecessor, aquele regista-se no servidor de nós.

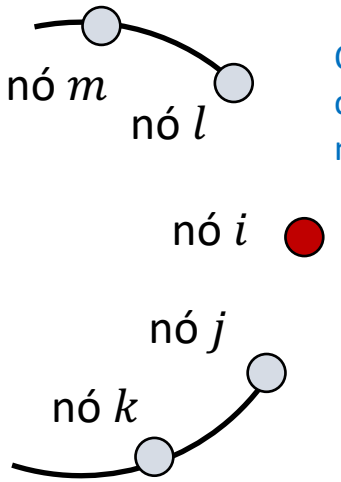
Camada Topológica: Saída do nó i do anel r

Saída do nó i



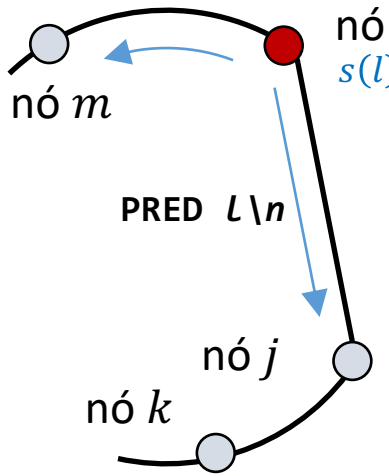


O nó i começa por retirar o seu registo do servidor de nós.

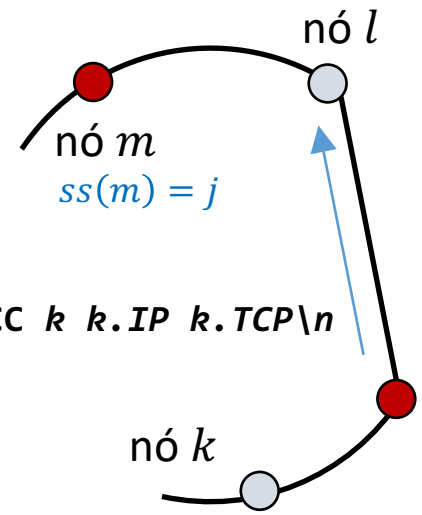


O nó i fecha as adjacências com as suas cordas, com o nó j e com o nó l .

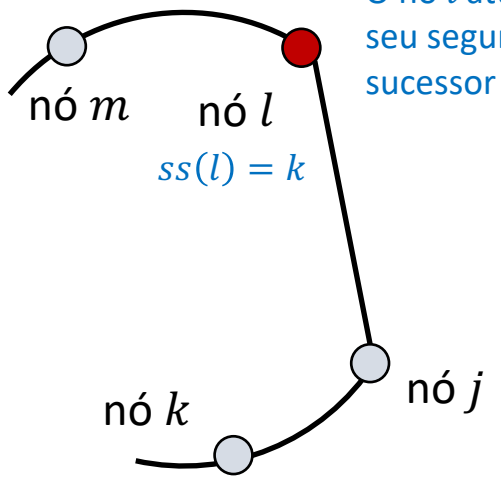
SUCC j j.IP j.TCP\n



O nó l abre uma adjacência com o nó j, promove-o a sucessor, informa-o que será seu predecessor e informa o seu predecessor da existência do nó j .



O nó j atualiza o seu predecessor para l, informa o nó l da existência do nó k . Concorrentemente, o predecessor do l atualiza o seu segundo-sucessor para j .



O nó l atualiza o seu segundo-sucessor para k.

Camada Topológica: Criação de uma corda

As cordas são estabelecidas por indicação do utilizador através do comando

`chord (c)`

O nó em causa consulta o servidor de nós e escolhe um outro nó, que não o sucessor nem o predecessor, para com ele estabelecer uma adjacência

Após estabelecimento da corda, o nó envia a seguinte mensagem ao outro extremo da corda

`CHORD i<LF>`

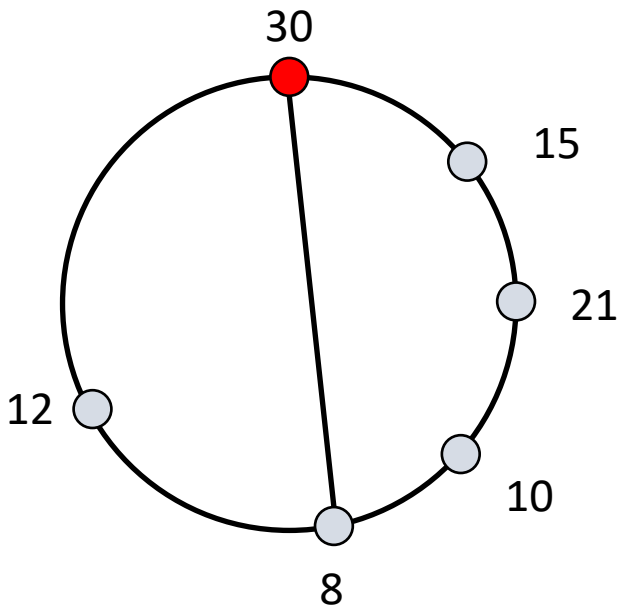
em que **`i`** identifica o nó que estabelece a corda

Uma corda pode ser removida com o comando

`remove chord (rc)`

Cada nó origina no máximo uma corda podendo ser recetor de várias.

Camada de encaminhamento: caminhos mais-curtos



- Para efeitos de encaminhamento, cada nó mantém uma *tabela de encaminhamento* indexada por nó destino e por vizinho que indica qual o caminho do nó ao nó destino através do vizinho.
- A partir da tabela de encaminhamento, cada nó determina uma *tabela de caminhos mais-curtos* e uma *tabela de expedição*, ambas indexadas por nó destino.
- A tabela de caminhos mais-curtos indica o caminho mais-curto para chegar a cada destino (casos de empate são quebrados arbitrariamente). Sempre que ela é alterada, o nó anuncia as alterações aos seus vizinhos através de mensagens de encaminhamento.
- A tabela de expedição indica o vizinho ao longo do caminho mais-curto até cada um dos destinos. Ela direcionará as mensagens da aplicação *chat*.

Tabela de encaminhamento do nó 30

	8	12	15
8	**30-8**	30-12-8	30-15-21-10-8
10	**30-8-10**	30-12-8-10	30-15-21-10
12	30-8-12	**30-12**	-
15	-	-	**30-15**
21	30-8-10-21	30-12-8-10-21	**30-15-21**
30	-	-	-

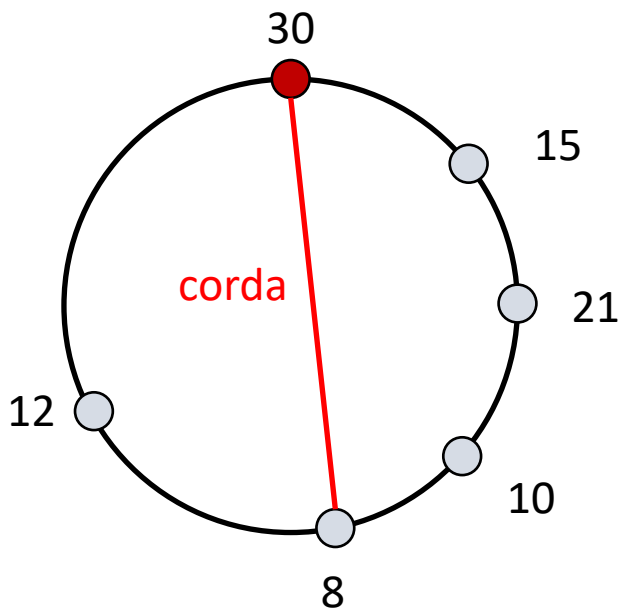
Tabela de caminhos mais curtos do nó 30

8	30-8
10	30-8-10
12	30-12
15	30-15
21	30-15-21
30	30

Tabela de expedição do nó 30

8	8
10	8
12	12
15	15
21	15
30	-

Camada de encaminhamento: Adição de uma adjacência



Quando uma adjacência é adicionada entre dois nós, cada um deles envia ao outro uma mensagem de encaminhamento por cada entrada na tabela de caminhos mais-curtos. Inicialmente, um nó que acabe de entrar na rede só se tem a si próprio como destino na sua tabela de caminhos mais-curtos.

Exemplo: criação da corda 30-8

Quando a adjacência 30-8 é estabelecida, o nó 30 envia ao nó 8, e apenas a este, as entradas da sua tabela de caminhos mais-curtos, e o nó 8 envia ao nó 30 as suas.
Quando o nó 30 recebe do nó 8 as entradas da sua tabela de caminhos mais-curtos, ele:
(i) re-calcula o seu caminho mais curto para o nó 8 como 30-8 e envia-o a todos os vizinhos;
(ii) re-calcula o seu caminho mais curto para o nó 10 como 30-8-10 e envia-o para todos os seus vizinhos.

Tabela de caminhos mais curtos do nó 30
(antes da criação da corda)

8	30-12-8
10	30-12-8-10
12	30-12
15	30-15
21	30-15-21
30	30

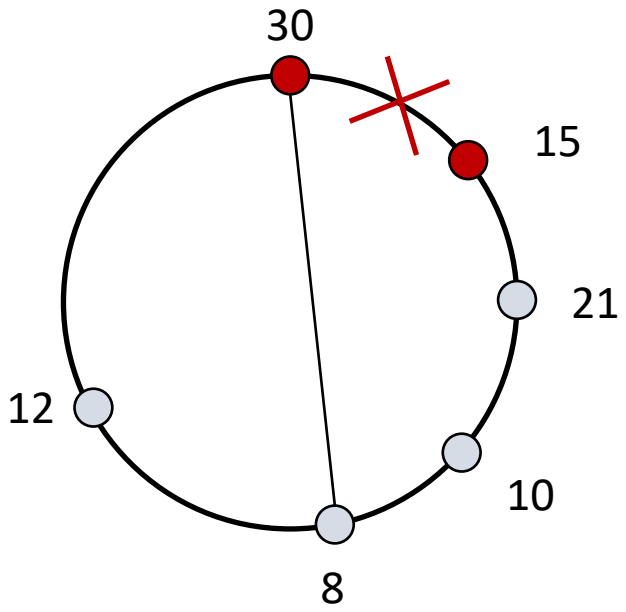
Mensagens de encaminhamento enviadas inicialmente pelo nó 30 ao nó 8 na sequência da adição da adjacência 30-8:

ROUTE *30 8 30-12-8<LF>*
ROUTE *30 10 30-12-8-10<LF>*
ROUTE *30 12 30-12<LF>*
ROUTE *30 15 30-15<LF>*
ROUTE *30 21 30-15-21<LF>*
ROUTE *30 30 30<LF>*

Tabela de caminhos mais curtos do nó 30
(depois da criação da corda)

8	30-8
10	30-8-10
12	30-12
15	30-15
21	30-15-21
30	30

Camada de encaminhamento: Remoção de uma adjacência



Quando uma adjacência entre dois nós é removida,

- cada um deles remove as entradas indexadas pelo outro da sua tabela de encaminhamento,
- atualiza a tabela de caminhos mais-curtos,
- anuncia as entradas alteradas desta aos seus vizinhos em mensagens de encaminhamento e
- atualiza a sua tabela de expedição.

Tabelas de encaminhamento do nó 30

	8	12	15
8	**30-8**	30-12-8	30-15-21-10-8
10	**30-8-10**	30-12-8-10	30-15-21-10
12	30-8-12	**30-12**	-
15	-	-	**30-15**
21	30-8-10-21	30-12-8-10-21	**30-15-21**
30	-	-	-

Mensagens de encaminhamento enviadas aos vizinhos:

ROUTE *30 15<LF>*
ROUTE *30 21 30-8-10-21<LF>*

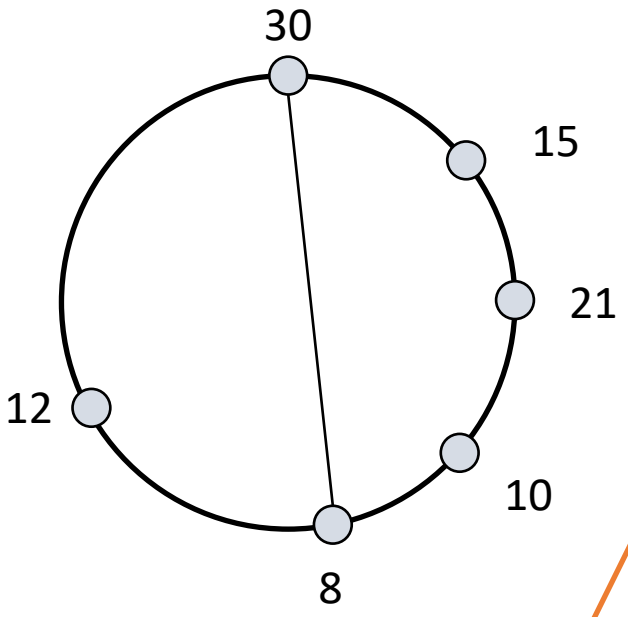
Tabelas de encaminhamento do nó 15

Nó 15	21	30
8	15-21-10-8	**15-30-8**
10	**15-21-10**	15-30-8-10
12	15-21-10-8-12	**15-30-12**
15	-	-
21	**15-21**	-
30	-	**15-30**

ROUTE *15 8 15-21-10-8<LF>*
ROUTE *15 12 15-21-10-8-12<LF>*
ROUTE *15 30<LF>*

Camada de encaminhamento:
Receção de uma mensagem de encaminhamento

Exemplo: **ROUTE *30 10 30-8-10<LF>***
(por TCP)



Uma mensagem de encaminhamento é da forma (i,n,ϕ) em que ϕ é um caminho com origem em i e destino em n. Quando o nó j recebe do vizinho i a mensagem de encaminhamento (i,n,ϕ), ele toma as seguintes ações:

- Calcula o caminho para n através do vizinho i. Se j é um nó de ϕ, então não há caminho válido para n através de i. Caso contrário, se j não é um nó de ϕ, então o caminho de j para n através de i é a concatenação de j com ϕ, $j{-}\phi$.
- Atualiza a entrada indexada por n e i na tabela de encaminhamento com o cálculo anterior.
- Atualiza a entrada indexada por n na tabela de caminhos mais-curtos.
- Se, efetivamente, a nova entrada na tabela de caminhos mais curtos for diferente da anterior, ela é anunciada a cada um dos vizinhos em mensagens de encaminhamento.
- Atualiza a entrada indexada por n na tabela de expedição.

Exemplo:
O nó 15 anuncia **ROUTE *15 12 15-30-12<LF>***
Esta alternativa não é um caminho válido para o nó 30.

Protocolo de *Chat* - Envio de mensagens

As mensagens de *chat* são enviadas por indicação do utilizador através do comando

`message (m) dest message`

em que ***`dest`*** identifica o nó de destino e ***`message`*** a mensagem de *chat* a enviar.

O envio da mensagem de *chat* é feito através do protocolo de *chat* com a mensagem

CHAT *`i n message<LF>`*

em que, ***`i`*** identifica o nó de origem, ***`n`*** identifica o nó de destino, e ***message*** a mensagem de *chat*.

As mensagens de *chat* têm 128 carateres no máximo.